Milestone 6, Part 4b: A Precision Bug, and the Blueprint in the Browser

holdem-cfr walkthrough

The result that did not fit

The arena from Part 4a compares checkpoints head to head. On the MacBook Air, three checkpoints of the same trainer gave these results:

Match	Result (mbb/hand)	Hands
924M iterations vs 408M iterations	**+39 ± 16**	800,000
408M iterations vs 34M iterations	**−36 ± 18**	800,000

The first line is what training should do. The second is not: twelve times more training produced a strategy that loses to the short run, and the interval excludes zero. Monte Carlo CFR is noisy, but its average strategy does not get *worse* with more iterations in a way that shows up at four standard errors. Something in the arithmetic was wrong.

Diagnosis: f32 sums stop absorbing updates

The trainer stores two numbers per (node, bucket, action): the cumulative regret, which drives the current strategy through regret matching, and the strategy sum, whose normalization is the average strategy that the blueprint actually plays. Both were 32-bit floats, updated with lock-free compare-and-swap adds.

A 32-bit float has a 24-bit significand. Between 2^{23} and 2^{24} (about 8.4 and 16.8 million) consecutive floats are exactly 1 apart; above 2^{24} they are 2 apart. The strategy sum at a node grows by the acting player's probability of each action, a number between 0 and 1, every time the node is visited. At a busy preflop node, with Linear-CFR discounting switched off after the first five minutes, the sum passes 2^{23} after a few hundred million iterations. From then on, adding 0.3 does nothing: 8,400,000.3 rounds back to 8,400,000. Adding 0.7 rounds up to 8,400,001.

So the average strategy at those nodes quietly turns into a *majority vote*: every visit adds 1 to whichever action the current strategy favours above 50% and nothing to the others. Mixed strategies, which are the whole point of an equilibrium and matter most preflop, are rounded toward pure ones.

The checkpoints confirm it directly. For each entry, the gap between adjacent floats at the entry's magnitude says how small an increment it can still absorb:

Checkpoint	Largest strategy sum	Entries that cannot absorb +0.01	Entries that cannot absorb +0.1
34M (VM, 6 min)	5.3×10^5	0.7%	0.00%
408M (Mac, 20 min)	6.0×10^6	7.9%	1.5%
924M (Mac, 40 min)	1.3×10^7	11.1%	3.4%

The damaged entries are the most visited ones, so they carry far more weight in play than their share of the table suggests. The 924M checkpoint still beat the 408M one because the regrets, which drive learning, stayed usable, and most of the table was still precise. But the curve was bending the wrong way.

The fix

Regrets and strategy sums are now 64-bit floats. The same compare-and-swap add works on `AtomicU64`, and a 53-bit significand pushes the "cannot absorb 0.3" threshold past 10^{15}, far beyond any run this project will make. The table doubles from 5.7 MB to 11 MB, which still fits in cache, and training speed on the VM is unchanged at roughly 90,000 iterations per second.

A unit test pins the failure down: it adds 0.3 to a float at 2^{24} a thousand times, checks that the f32 sum has not moved, and checks that the f64 sum has moved by exactly 300.

Checkpoints move to format 2 (`HCFRBP02`, f64). Format 1 files still load, converted on read, and a test writes a format 1 file by hand to prove it. The new `--reset-average` flag zeroes the strategy sums and keeps the regrets. The regrets of a long run are its most valuable part, and restarting the average from them discards both the f32 damage and the early, poor iterations, which is the same reason Linear CFR weights late iterations more.

Putting the blueprint in a browser

The web table's 6-max bots are heuristics. The trained blueprint is a two-player strategy, so it gets its own heads-up mode, and the interesting problem is the card abstraction. The blueprint's strategy is indexed by bucket, and the bucket tables are 138 MB, far too big for a page.

The page therefore **recomputes** each bucket from the cards, using the definitions the tables were built from:

- **River:** hand strength HS = (wins + ties/2) / 990 against every opponent hand, rounded through the u16 quantization the river table uses. - **Turn:** E[HS²], the mean of HS² over the 46 possible rivers. - **Flop:** E[HS²], the mean of the turn values over the 47 possible turns. Each five-card board appears twice (turn *t* then river *r*, and the reverse), so a cache halves the work to 1,081 river boards of 991 evaluations each.

Getting the *same bucket* requires the same number, not a close one, because a value a hair below a threshold lands in the next bucket down. So the JavaScript copies the Rust code's arithmetic exactly. It casts to f32 (`Math.fround`) wherever Rust casts to `f32`, sums in f64 in the same card order, and rounds the same way. The thresholds are not stored in the bucket file, so `export-web` recomputes them from the strength tables and refuses to export unless they reproduce every one of the 138 million entries of the bucket file. They did, on all three streets.

Speed comes from a second evaluator, written for this purpose. It tracks the ranks present once, twice, three and four times as four 13-bit masks and folds in two hole cards without allocating. Lookup tables give popcounts, highest bits and straights. It returns exactly the values of the page's reference evaluator, checked on 200,000 random hands, and computes a flop bucket in about 50 ms.

The checks, run in CI:

- On 300 random deals exported from Rust, all 660 street metrics are **bit-identical** to the Rust tables, and all 660 buckets match. The test allows a one-ulp difference, because Rust computes each isomorphism class from a canonical suit labelling that sums in a different order, but none occurred. - The exported tree has 13,704 nodes, 5,526 action nodes and 707,641 strategy entries. Every strategy row sums to 1. - Replaying 20,000 random lines reproduces the tree's chip counts at every terminal, every payoff is bounded by the chips in, and every showdown has equal stakes.

After the fix: head-to-head runs out of resolution

On the Mac, the 924M checkpoint was loaded into the f64 trainer with `--reset-average` and trained for 30 more minutes, to 1.552 billion iterations. Its average strategy was rebuilt from 628 million clean iterations. Against the earlier checkpoints:

Match	Result (mbb/hand)	Hands
1,552M (f64, reset) vs 924M	−13 ± 17	800,000
1,552M (f64, reset) vs 408M	−44 ± 16	800,000
924M vs 408M (from before)	+39 ± 16	800,000

These three results cannot all reflect one ranking. 924M beats 408M, 1,552M ties 924M, and yet 1,552M loses to 408M. Head-to-head results between strategies that are all close to equilibrium are not transitive. Each match measures how one strategy's particular leaks line up against the other's, which is rock-paper-scissors at the scale of a few hundredths of a big blind, not a distance from equilibrium. Training has reached the point where checkpoint matches cannot rank checkpoints any more.

The measurement that can is exploitability. It cannot be computed exactly in the full game, but **local best response** (Lisý and Bowling, 2017) gives a lower bound. At every decision it plays the best action against the blueprint's actual range, assuming the hand then checks down, and it is the standard yardstick for no-limit bots. It is next on the roadmap, together with real-time re-solving.

Exported independently on the Mac and on the VM, the bucket thresholds and the fixture deals came out byte-identical.

The heads-up mode

You play the blueprint with blinds of 0.5/1 bb and both stacks reset to 100 bb each hand, swapping the button every hand. Your options are the blueprint's own menu: one open size, 33% or 75% pot on the flop, 66% on the turn, 66% or 125% on the river, pot-sized raises, and all-in. That way it never has to interpret a bet it was not trained on, which is what action translation, the next part, will handle.

After each hand the panel shows the blueprint's reasoning at every decision: its bucket, the $E[HS^2]$ or HS value, and the probability it gave each action before sampling one. With *Show bot cards* on, this appears live. *Ask the blueprint* shows what the blueprint would do with your hand. The results tab tracks bb/100 with a 95% interval, because heads-up results take thousands of hands to mean anything.

What the bot does **not** do is just as important. It plays the raw average strategy of an abstract game, with no real-time search. It cannot tell apart two hands in the same bucket, and on the turn it no longer remembers its flop bucket. Libratus and Pluribus fix exactly these weaknesses with real-time subgame solving, which is next on the roadmap: re-solve from the turn with the postflop solver of Milestones 4 and 5, starting from ranges taken from the blueprint.

Reproduce

```
# Rebuild the average from the trained regrets with f64 sums:
cargo run --release -p cli -- blueprint --minutes 30 --threads 6 --reset-average
cp data/blueprint.bin data/bp_f64.bin
cargo run --release -p cli -- arena --vs data/bp_408M.bin --pairs 400000
# Export to the browser (rebuilds the strength tables, ~1 min on 8 cores):
cargo run --release -p cli -- export-web
node web/table/blueprint.test.js
python3 web/table/build.py   # then open web/table/index.html
```